Software Requirements Specification

Code Quality Checker

Software Construction & Development Course

1. Introduction

1.1 Purpose

This document describes the requirements for a Code Quality Checker — a tool that reads source code and gives developers useful feedback about its quality, style, and complexity.

The goal is simple: help developers spot bad habits, write cleaner code, and build software that is easier to maintain and understand.

This project is developed as part of the Software Construction & Development course.

1.2 Scope

The Code Quality Checker will:

- Accept source code as input
- Analyze the code using a set of predefined quality rules
- Detect common bad coding practices
- Calculate a quality score between 0 and 100
- Return clear warnings and improvement suggestions

The system is built using the following technologies:

- Backend: Java Spring Boot
- Frontend: HTML, CSS, and JavaScript
- Communication: REST API

1.3 Definitions

Term	Description
Rule Engine	The component that applies quality rules to the submitted code.
Score	A number from 0 to 100 representing how good the code quality is.

Warning	A message shown to the user pointing out a poor coding practice.
API	The interface that connects the frontend and the backend.

2. Overall Description

2.1 Product Perspective

The system is made up of three main parts that work together:

- A frontend UI where the user types or pastes their code
- A Spring Boot REST API that receives the code and processes it
- A rule-based analysis engine that evaluates the code and produces a result

The overall flow looks like this:

```
User → Frontend → Spring Boot API → Rule Engine → Result → Frontend
```

2.2 Product Functions

At a high level, the system will:

- Accept code input from the user
- Run the code through multiple quality rules
- Generate a quality score
- Return a list of warnings and suggestions
- Display everything clearly on the frontend

2.3 User Classes

User Type	Description
Developer (Primary)	Submits source code, reviews the quality results, and uses the feedback to improve their code.
User Type	**Description**

Instructor (Secondary)	Reviews the project to evaluate whether the system works as expected.

2.4 Operating Environment

The system is designed to run in the following environment:

- Java 17 or higher
- Spring Boot framework
- Any modern web browser (Chrome, Edge recommended)
- Supported operating systems: Windows, Linux, macOS

3. System Features

3.1 Code Input

The user can type or paste source code into a text area on the frontend. Once submitted, the code is sent to the backend for analysis.

- Input: Java source code entered as plain text
- Output: The code is forwarded to the backend API

3.2 Code Analysis

The backend analyzes the submitted code using a set of predefined rules. The current checks include:

- Detection of debug print statements (e.g., System.out.println)
- Identification of poor variable names (e.g., single-letter names like x, y, a)
- Detection of high code complexity
- Flagging of other common bad coding practices

3.3 Score Generation

After analysis, the system calculates a quality score from 0 to 100. A higher score means better code quality. For example, a score of 85 means the code is generally good but has a few issues worth fixing.

3.4 Warning Generation

For every issue found, the system generates a warning message that tells the user what is wrong and how to fix it. Example warnings include:

- "Avoid using System.out.println in production code"
- "Use meaningful variable names instead of single letters"
- "High complexity detected — consider breaking this into smaller methods"

4. Functional Requirements

ID	Requirement
FR1	The system shall accept source code as text input from the user.
FR2	The system shall analyze the submitted code using the defined rule set.
FR3	The system shall calculate and return a quality score from 0 to 100.
FR4	The system shall return a list of warnings for each issue found.
FR5	The system shall expose a REST API endpoint to handle analysis requests.
FR6	The system shall display the score and warnings on the frontend.

5. Non-Functional Requirements

Category	Requirement
Performance	The system should return analysis results in under 2 seconds.
Usability	The UI should be clean and simple — code input and results should be easy to understand at a glance.

Maintainability	The rule engine should be modular so that individual rules can be updated or replaced without affecting other parts of the system.
Scalability	New quality rules should be easy to add without changing the core system architecture.

6. System Architecture

The system follows a layered architecture. Each layer has a single responsibility and communicates only with the layer directly below it:

```
Frontend (HTML/CSS/JS)
  ↓
Controller (REST API Layer)
  ↓
Service (Business Logic)
  ↓
Rules Engine (Code Analysis)
  ↓
Result Model (Score + Warnings)
```

7. Future Enhancements

The following features are not in scope for the current version but could be added in the future:

- File upload support — allow users to upload .java files directly instead of pasting code
- Multiple language support extend analysis beyond Java to other languages like Python or C++
- AI-based analysis use a language model to provide smarter, context-aware suggestions
- UI dashboard show score history, trends, and comparisons over time
- PDF report generation let users download a formatted analysis report

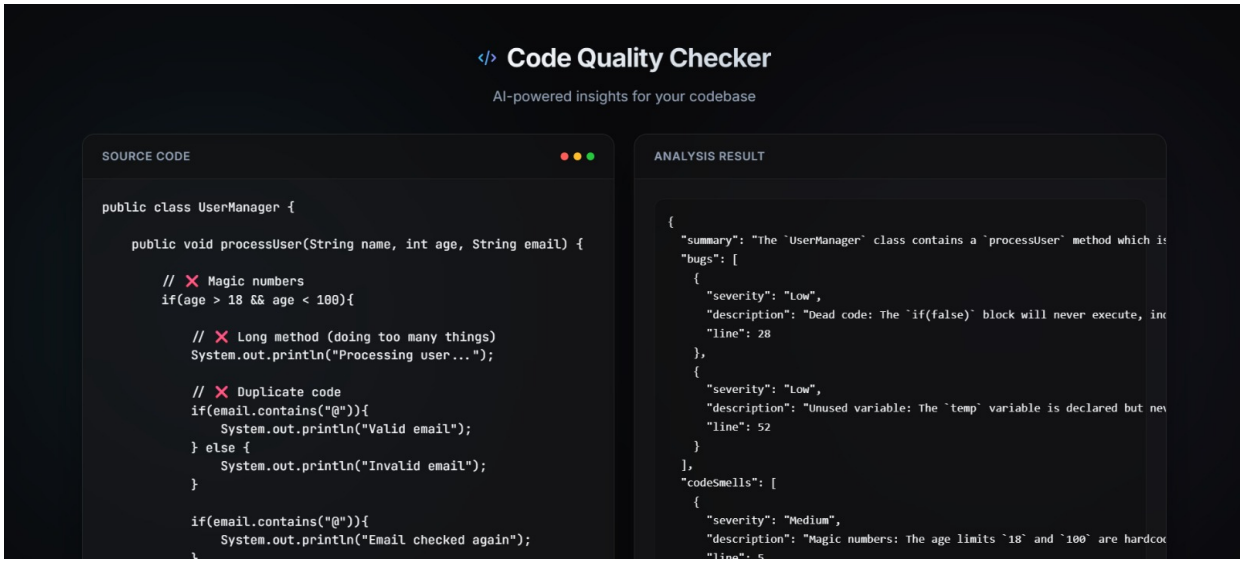
8. System Screenshots

This section presents the screenshots of the implemented Code Quality Checker system. These screenshots demonstrate the functionality of the application and show how different components of the system work together.

The screenshots included in this section represent important parts of the project implementation, including the frontend result page, backend execution, and API communication testing.

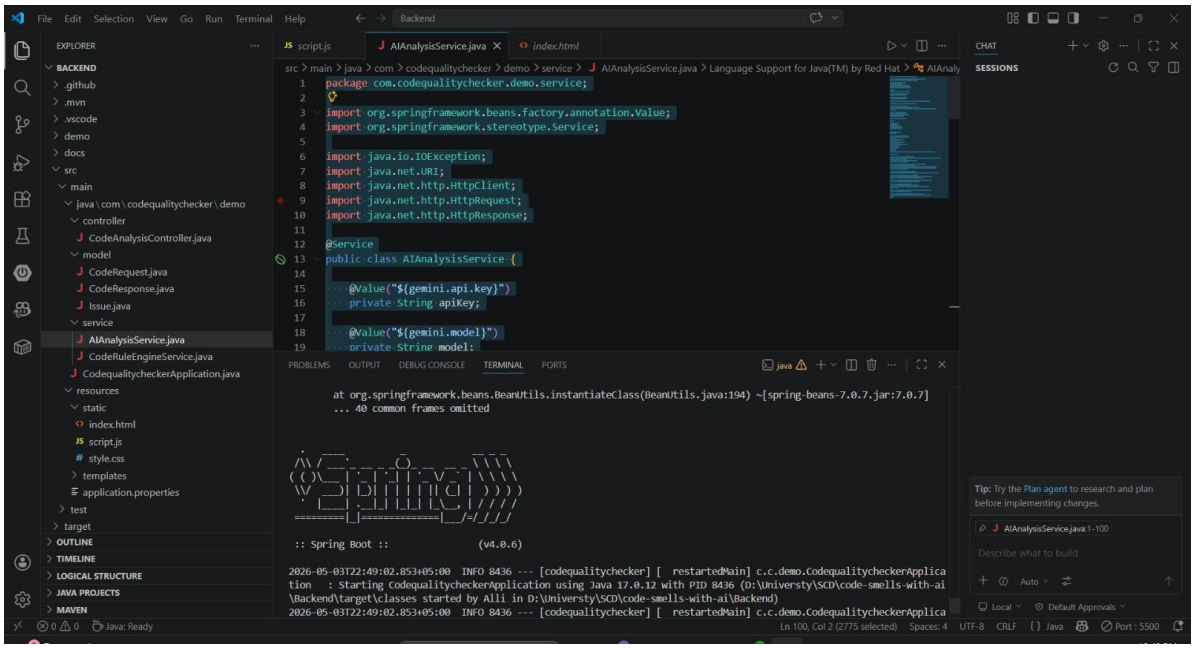
Included Screenshots

- **Result Page:**



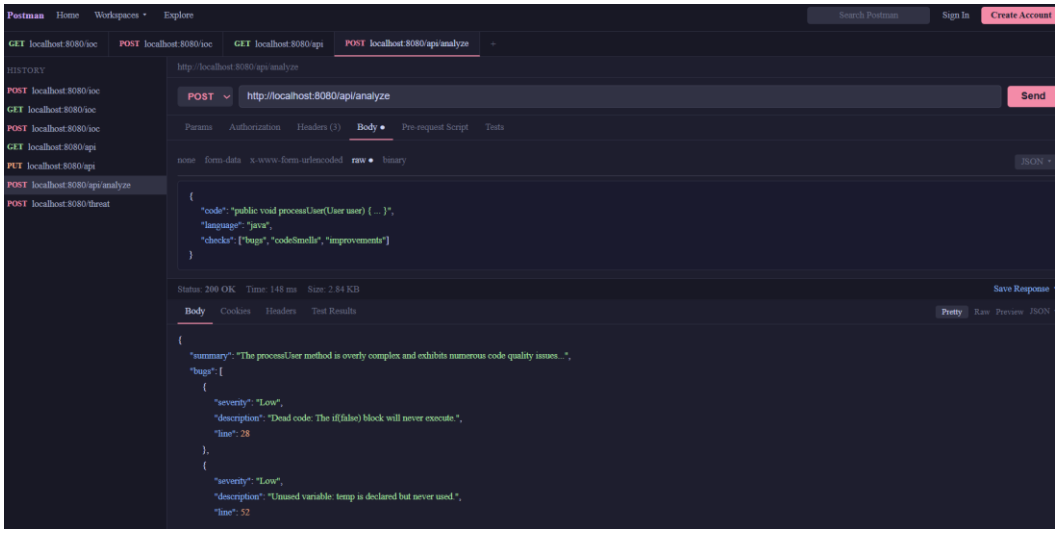
Displays the analysis results generated by the system, including code quality feedback, warnings, suggestions, and quality score returned after code analysis.

- **Java Backend Execution:**



Shows the Spring Boot backend server running successfully, confirming that the backend services and application logic are functioning properly.

- **Postman API Testing (Localhost):**



Demonstrates successful API communication between the client and backend server using localhost endpoints. This screenshot verifies that the REST API is correctly processing requests and returning responses.

9. Conclusion

The Code Quality Checker is a lightweight but practical tool that helps developers write better code. By combining a rule-based analysis engine with a simple REST API and a clean frontend, the system makes it easy for anyone to get immediate feedback on their code quality. The modular design also ensures that the system can grow over time — new rules can be added, new languages can be supported, and more advanced features can be built on top of the existing foundation.